CASSANDRAGARGOYLE · EKOSYSTÉM PORTUNIX

VIM

Vrstvený informační model: jak se dokument učí říct, co znamená

Komunita CassandraGargoyle

Verze 1.1.0 · Sestaveno 2026-10-25



Obsah

ČÁST 1 **Prolog** 3

1 Kdo je zákazník? ... 4

ČÁST 2 **Jak VIM rostl** 7

2 Skleněné tabule ... 8

3 Od 1.0 k 2.1 ... 11

ČÁST 3 **Uvnitř VIM** 15

4 Uvnitř vim.json ... 16

5 Stroj, který čte ... 22

6 Slib v JSON Schema ... 26

ČÁST 4 **Práce s VIM** 30

7 Váš první vim.json ... 31

8 Kam VIM směřuje ... 35

ČÁST 1

Prolog

Jedna otázka nad jednou fakturou, položená dvakrát: jednou prostému textu, jednou vrstvenému modelu.

1 · Kdo je zákazník?

KAPITOLA 1

Kdo je zákazník?

Jedna otázka, položená dvakrát

Poslední lednový den roku 2026 položil někdo z týmu jazykovému modelu velmi jednoduchou otázku nad vyúčtováním elektřiny: *kdo je na tomto dokumentu zákazník?*

Člověku stačí na odpověď půl vteřiny. Jméno stojí přímo tam, pod slovem *Zákazník*, v levém horním rohu stránky. Vedle, ve druhém sloupci, je slovo *Adresát* a pod ním totéž jméno a tatáž adresa. Každé dítě, které umí číst, by ukázalo na správný blok.

Model dostal dokument jako prostý text, tak jak ho většina nástrojů z PDF dodnes předává: řádek za řádkem, od horního okraje stránky k dolnímu. Chvíli četl a pak odpověděl poctivě. Zákazníka podle něj *nelze s jistotou určit*. Existuje kandidát, jakýsi Ing. Jan Novák, ale vztah k popisku není výslovný. První řádek textu zněl „*Zákazník Adresát*“: dvě záhlaví sloupců se slila do jednoho řádku. O sto řádků níž se jméno objevilo dvakrát na jednom řádku a adresa také. Dva sloupce stránky se zploštily do jednoho proudu slov. Jistota odpovědi: **nízká**.

O několik minut později dostal tentýž model tutéž otázku nad tímtéž vyúčtováním. Tentokrát přišel dokument jako soubor `vim.json`. Odpověď byla krátká: Ing. Jan Novák, s adresou, a jistota **vysoká**. Model k tomu vysvětlil proč. V souboru byl blok typu `labeled_content` se záhlavím *Zákazník* a se jménem pod ním a vedle samostatný blok se záhlavím *Adresát*. O vrstvu výš ležela entita typu `customer`, která ukazovala zpět na první blok. Nic se nemuselo hádat.



Obrázek 1.1 Totéž dvousloupcové záhlaví jako prostý text a jako dva popsané bloky strukturální vrstvy

Oba výsledky jsou dodnes uložené v repozitáři parseru vedle sebe jako první pokus malé výzkumné aparatury. Nejsou benchmarkem. Jsou jednou otázkou nad jedním anonymizovaným dokumentem. Na několika řádcích ale ukazují celý důvod, proč tato kniha vznikla.

Problém není v PDF

Svalit vinu na formát je snadné. PDF je všude a proslulo tím, že se strojově čte špatně. PDF ale nikdy nemělo nést význam. Adobe ho uvedlo v roce 1993, aby stránka vypadala stejně na každé obrazovce a na každé tiskárně, a v roce 2008 se z něj stala mezinárodní norma ISO 32000. Tuto práci dělá velmi dobře. PDF ví, kde stojí každý znak. Neví, že znaky pod slovem *Zákazník* tvoří jméno, ani že to jméno patří zákazníkovi, a ne adresátovi.

Návrhové poznámky VIM to říkají přímo: *PDF je jen symptom*. Skutečným problémem je **ztráta významu, když data putují mezi systémy**. Typický řetěz vypadá takto:

```
DB → reporting tool → PDF → human → manual transcription
```

Na začátku řetězu jsou data dokonalá: databáze přesně ví, která hodnota je zákazník, která částka a která měna. Pak je reportovací nástroj vytiskne na stránku, stránka putuje k někomu jinému a na druhé straně musí člověk, nebo dnes program, význam z obrázku znovu poskládat. Příjemce přitom nechce PDF. Chce hodnoty, vztahy, význam a důkaz, že hodnoty skutečně pocházejí z tohoto dokumentu.

Odpovědí není jeden formát

Když se na tento problém díváme poprvé, toužíme po jednom dokonalém formátu, který pojme všechno. Historie dokumentů říká, že takový neexistuje. Knihovny se se stejnou otázkou potkaly, když začaly digitalizovat knihy: v červnu 2004 vydal evropský projekt METAe první verzi formátu ALTO, XML, které zaznamenává, kde na naskenované stránce stojí každé slovo, řádek a sloupec. ALTO popisuje rozvržení výborně a Kongresová knihovna ho spravuje dodnes. O tom, co slovo znamená, ale neříká nic. Jiné formáty zase výborně popisují význam a o stránce nevědí nic.

Návrhové poznámky VIM z toho vyvozují závěr ve dvou větách, kterými celý koncept začíná:

> Neexistuje jeden univerzální formát pro všechny soubory. Existuje univerzální vrstvený model pro reprezentaci informace.

Otázka tedy nezní, *který formát uloží všechno*. Zní, *jak uložit výsledek porozumění souboru*. Odpovědí VIM je stoh vrstev. Nejnižší vrstva říká, co na stránce fyzicky je. Další říká, jak je to seskupené. Třetí říká, co to znamená. Každá hodnota na každé vrstvě umí ukázat zpět na soubor, stránku a obdélník, ze kterého pochází, a říká, jak moc si byl stroj jistý.

Jméno

VIM je zkratka **Vrstveného informačního modelu**, anglicky *Layered Information Model*. Model se narodil v českém týmu, na českých fakturách a českých QR platbách, a jméno si svůj původ ponechalo. V angličtině písmena slovům neodpovídají, a tak kniha používá VIM jako vlastní jméno, podobně jako lidé říkají PHP nebo GNU a nepřemýšlejí, co písmena kdysi znamenala. Některé dokumenty ekosystému nějakou dobu rozepisovaly VIM jako *Versatile Information Model*. Takový rozpis zachoval písmena, ale ztratil podstatu: důležité slovo je *vrstvený*.

Jméno má ještě jednu malou past. Čtenář, který pracuje s textovými editory, si okamžitě vybaví editor Vim. Kromě tří písmen nemají nic společného. VIM z této knihy je datový model; jeho soubory končí příponou `.vim.json` a nikdo se nemusí učit, jak z něj odejít.

O čem je tato kniha

Kniha vypráví příběh VIM: jak se parser faktur, napsaný během několika horečných dní na konci ledna 2026, za devět měsíců proměnil ve společný kontejner výsledků celého ekosystému nástrojů pro dokumenty. Ukazuje, co obsahuje `vim.json`, vrstvu po vrstvě, na skutečném souboru ze skutečného parseru. Nahlíží do stroje, který ho píše, a na kontrakt v tomto repozitáři, který každému výrobci i každému čtenáři říká, co smí očekávat. A ukazuje, jak vyrobit první vlastní `vim.json` a jak ho zkontrolovat.

Další kapitola začíná samotnou myšlenkou: co znamená číst dokument ve vrstvách.

Prameny

- Výsledky pokusu: `research/results/` pluginu `vim-parser` v repozitáři `portunix-plugins` (31. ledna 2026)
- Návrhové poznámky *Universal "Perfect" Format for Storing Information from Any File*, `plugins/vim-parser/README.md` v `portunix-plugins`
- PDF: uvedeno firmou Adobe v roce 1993, ISO 32000-1 v roce 2008
- ALTO (XML): verze 1.0 z projektu METAe v červnu 2004, od roku 2009 ji spravuje Kongresová knihovna

ČÁST 2

Jak VIM rostl

Myšlenka vrstev a devět měsíců, během kterých se z parseru faktur stal společný model celého ekosystému.

2 · Skleněné tabule

3 · Od 1.0 k 2.1

KAPITOLA 2

Skleněné tabule

Čtení není jeden úkon

Pozorujte člověka, který poprvé čte fakturu. Oči neběží od prvního písmene k poslednímu. Nejdřív vezmou stránku jako obraz: tučný řádek nahoře, dva bloky textu vedle sebe, tabulka, černý čtverec v rohu. Pak seskupují: tento blok je dodavatel, tamten odběratel, tahle mřížka je seznam položek. Teprve potom přichází význam: číslo vpravo dole je celková částka, v korunách, a černý čtverec je QR kód, kterým se dá faktura zaplatit.

Čtení není jeden úkon. Jsou to nejméně tři a každý stojí na tom pod ním. Chceme-li, aby stroj výsledek čtení uchoval, je přirozené uchovat všechny tři a držet je od sebe. Představte si tři skleněné tabule položené přes stránku. Na nejspodnější obtáhnete, co tam opravdu je. Na druhé orámujete, co k sobě patří. Na horní napíšete, co to znamená. Každou tabuli lze zvednout a prohlédnout samotnou a každá značka na vyšší tabuli leží přesně nad značkami, ze kterých vznikla.

Tento obraz je VIM.

Pět principů

Návrhové poznámky VIM jmenují pět principů. Vyplatí se je číst pomalu, protože každé pozdější rozhodnutí o formátu se k některému z nich vrací.

Odděl obsah, strukturu a význam. Text `123 456` je obsah. To, že leží v buňce, ve sloupci, v tabulce, je struktura. To, že jde o částku bez DPH v korunách, je význam. Tři různé věci, uložené na třech různých místech.

Buď vrstvený. Jedna vrstva nikdy nestačí. Poznámky to shrnují krátkým řetězem: surová data, rozvržení, struktura, sémantika. Model, který uchová jen poslední krok, zahodí důkazy; model, který uchová jen první krok, nechá veškerou práci na čtenáři.

Uchovej původ. Každá hodnota musí umět říct, odkud pochází: z jakého souboru, z jaké stránky, z jaké oblasti stránky. Bez toho je hodnota jen pověst.

Buď obousměrný. Z modelu musí jít počítat, třeba sečíst částky faktur za celý rok. A z modelu musí jít dokument alespoň přibližně zrekonstruovat. Jedna z dalších kapitol ukazuje stránku faktury překreslenou jen z jejího `vim.json`.

Počítej s nejistotou. Žádný stroj nečte dokument se stoprocentní jistotou. Proto každý výsledek nese spolehlivost, metodu, která ho vytvořila, a tam, kde to dává smysl, i alternativy, které byly zváženy a odmítnuty.

Tři otázky

První tři vrstvy VIM odpovídají na tři otázky a jejich jména jsou dost krátká, aby se dala zapamatovat:

Vrstva	Otázka	Co obsahuje
`physical`	Co tam je?	Text s přesnou polohou a písmem, čárové kódy, tabulky, grafiku, plný text
`structural`	Jak to vypadá?	Bloky se záhlavím a tělem, jejich typ a důležitost, čárové kódy s bloky v okolí
`semantic`	Co to znamená?	Entity jako zákazník nebo dodavatel, platby přečtené z QR kódů, kmeny slov pro vyhledávání

Pořadí odpovídá hloubce interpretace. Fyzická vrstva neinterpretuje téměř nic: říká, že řetězec *Odběratel* stojí tučně na určitém místě. Strukturální vrstva interpretuje trochu: říká, že tento tučný řetězec je záhlavím bloku se třemi řádky pod ním. Sémantická vrstva interpretuje nejvíc: říká, že blok popisuje zákazníka, a říká to se spolehlivostí 0,8, ne 1,0.

Každá vyšší vrstva odkazuje na nižší stabilním identifikátorem, místo aby ji kopírovala. Strukturální blok `block_4` má stejné ID jako fyzický blok, ze kterého vznikl. Sémantická entita říká `source_block: block_4`. To je ta značka na horní skleněné tabuli, která leží přesně nad značkou pod ní.

Co se stalo se čtvrtou vrstvou

Původní návrhové poznámky měly čtvrtou vrstvu: *Původ a důvěra*, s otázkou *proč tomu věříme?* Měla obsahovat zdroj, stránku, obdélník, spolehlivost a metodu každé hodnoty.

Když se formát stavěl, samostatnou tabulí se tato vrstva nestala. Stalo se z ní něco lepšího: vlastnost každého prvku na každé tabuli. Textový prvek ve fyzické vrstvě nese svůj `bbox`, svou `page` a svou `confidence`. Strukturální čárový kód nese ID bloků ve svém okolí a spolehlivost. Sémantická entita nese ID zdrojového bloku a spolehlivost. Původ není místo, kam se chodí; cestuje s každou hodnotou. Spotřebitel nikdy nemusí spojovat dvě vrstvy, aby zjistil, odkud se číslo vzalo.

Vrstvy, které přišly později

Tři úrovně interpretace jsou uzavřená množina. Čtvrtá úroveň interpretace by změnila základní myšlenku VIM a architektonická rozhodnutí ekosystému to říkají výslovně. Na dokument se ale dá dívat nejen hlouběji, ale i z jiné strany. Postupně tak VIM dostal tři další vrstvy jiného druhu, **doménové vrstvy**:

Vrstva	Od verze	Otázka	Výrobce
`toc`	1.4	Jak v něm věci najdu?	vim-parser
`extraction`	2.0	Co nese tento formulář?	rozpoznávání formulářů (reco) a další
`prescriptive`	2.1	Co tento dokument předepisuje?	výrobce s jazykovým modelem, kontrolovaný člověkem

Doménová vrstva neinterpretuje stránku hlouběji. Drží pohled postavený pro jeden účel: orientaci v dlouhé knize, hodnoty vyplněného formuláře, pravidla zapsaná ve specifikaci. Každá doménová vrstva odkazuje na úrovně interpretace přes ID, takže kotvy zůstávají na jednom místě.



Obrázek 2.1 Šest vrstev VIM: dole tři úrovně interpretace, nahoře tři doménové vrstvy, které odkazují dolů přes ID

Všechny vrstvy jsou volitelné. Dokument ze skeneru obrázků může mít jen vrstvu `extraction`. Dlouhá kniha v PDF může mít `physical`, `structural` a `toc`. Čtenář vrstvy, které nezná, ignoruje. Právě to dovoluje velmi rozdílným nástrojům sdílet jeden kontejner, aniž by si šlapaly na prsty.

Model, ne formát souboru

V návrhových poznámkách je ještě jedna myšlenka, kterou je snadné přehlédnout. *Dokonalý formát*, říkají poznámky, není jedna věc, ale **pipeline plus datový model**. VIM je ten model. Určuje, jak vypadá výsledek porozumění dokumentu. Neurčuje, jak porozumění probíhá. Jeden nástroj čte text přímo z PDF, druhý pouští OCR na sken, třetí se zeptá jazykového modelu. Všechny píší stejné vrstvy a každá hodnota říká, jaká metoda ji vytvořila.

Proto také mohl VIM růst tak rychle. Jeho první verze vznikla pro faktury v PDF. Během několika měsíců nesl výsledky rozpoznávání formulářů z naskenovaných obrázků i požadavky technických specifikací. Model nemusel měnit svou myšlenku; stačilo mu přidat nové skleněné tabule. Jak se to stalo a v jakém pořadí, vypráví další kapitola.

Prameny

- Návrhové poznámky *Universal "Perfect" Format for Storing Information from Any File*, `plugins/vim-parser/README.md` v `portunix-plugins`
- Specifikace formátu VIM v1.5 a v2.0, `docs/architecture/` v `portunix-architecture`
- ADR-009 *Criteria for Adding a New VIM Layer* (9. října 2026), `portunix-architecture`

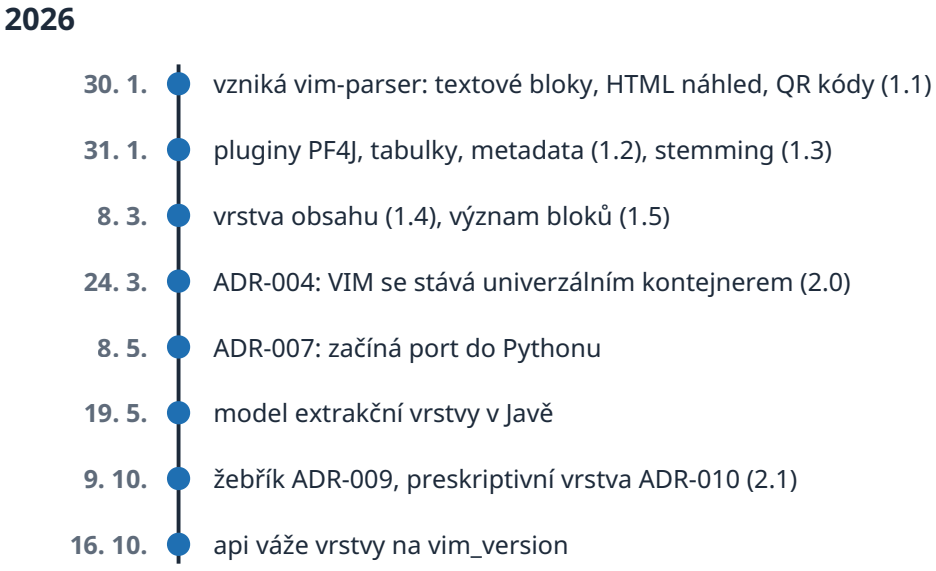
KAPITOLA 3

Od 1.0 k 2.1

Dva dny v lednu

Většina formátů se rodí pomalu, ve výborech, přes návrhy a připomínky. VIM se narodil ve spěchu. V pátek 30. ledna 2026 se v repozitáři pluginů Portunixu objevil nový plugin. Jeho první commit se jmenoval *Initial plugin scaffold with VIM architecture*. Do večera téhož dne uměl plugin rozřezat text PDF na bloky s přesnou polohou, ukázat výsledek jako HTML stránku a najít na stránce dvourozměrné čárové kódy. Formát měl první dvě verze: **1.0** s fyzickou a strukturální vrstvou a **1.1** s čárovými kódy.

Další den, sobota, byl ještě rušnější. Plugin se rozdělil do modulů a dostal vlastní systém pluginů postavený na javové knihovně PF4J, aby každý druh vytěžování mohl žít ve vlastním balíčku a dal se přidat nebo odebrat bez nového sestavení jádra. Orchestrátor se naučil spouštět pluginy sám ve správném pořadí. Přibyly tabulky, pak metadata dokumentu a plný text (**1.2**), pak obrázky a večer lemmatizátor, který převádí slova na kmeny pro vyhledávání (**1.3**). Téhož dne proběhl pokus z prologu: otázka *kdo je zákazník* položená prostému textu a souboru `vim.json`. Vrstvený model prošel první zkouškou, když mu byl sotva den.



Obrázek 3.1 Rok 2026 ve VIM: od prvního commitu parseru po kontrakt verze 2.1

Dlouhý ocas skutečných dokumentů

Po prvním spěchu přišla pomalá práce, kterou zná každý nástroj na dokumenty: skutečné dokumenty. Začátkem února dostal lemmatizátor nativní backendy pro češtinu, napsané v C++ a připojené přes JNI, protože čeština je jazyk s bohatým tvaroslovím a jednoduchý stemmer jí nestačí. Detekce tabulek se přestěhovala do vlastního univerzálního pluginu, aby ji mohly využít i jiné nástroje.

Na konci února odhalil článek o umělé inteligenci typografický detail: **iniciálu**, velké první písmeno odstavce. Pro stroj, který seskupuje znaky podle polohy, je iniciála osamělé obří písmeno, které nepatří do žádného řádku. Parser se ji naučil rozpoznat a připojit zpět ke slovu, které začíná, a textový prvek dostal pole `drop_cap_merged` a `drop_cap_bbox`. Tak VIM obvykle roste: ne velkými plány, ale jedním tvrdohlavým dokumentem po druhém.

V březnu se parser potkal s knihami. Skript stáhl ukázkové knihy, mezi nimi *Hovory k sobě* Marka Aurelia, a tým naučil parser číst obsah knihy, jak ten vytištěný, tak ten odvozený z nadpisů. Dne 8. března se z toho stala vrstva `toc` a verze **1.4**. Téhož dne dostaly bloky **důležitost**, primární nebo pomocnou, aby číslo stránky nebo živé záhlaví už nevypadaly stejně důležitě jako text, a výchozí verzí se stala **1.5**.

Malá zvláštnost z té doby je vidět dodnes. V řadě 1.x nastavuje parser `vim_version` podle funkcí, které dokument skutečně používá. Dokument s obsahem říká `"1.4"`, přestože parser zná verzi 1.5. V letech 1.x byla `vim_version` blíž seznamu funkcí než verzi kontraktu. Verze 2 to změnila.

Kontejner pro všechny

Dne 24. března 2026 změnilo jedno architektonické rozhodnutí roli VIM. Jeho název byl krátký: *VIM as Universal Result Container* (ADR-004).

Příčinou byl nový výrobce. Ekosystém Portunix dostával framework pro rozpoznávání formulářů, `portunix-reco`, napsaný v Pythonu. Čte naskenované obrázky, ne PDF. Jeden logický dokument může pocházet z mnoha obrázků. Vytěžuje pole z předem daných zón formuláře a tabulka se může táhnout přes několik stránek a musí se spojit v jednu. VIM 1.5 nic z toho neuměl: jeho fyzická vrstva předpokládala text z PDF s písmy a sémantická vrstva znala pevnou sadu typů entit.

Rozhodnutí zachovalo vše, co existovalo, a přidalo, co chybělo. Čtyři existující vrstvy zůstaly beze změny; dokument 1.5 zůstal platným dokumentem 2.0. Nová vrstva `extraction` přijala výsledky zpracování formulářů: odkaz na šablonu, která vytěžování řídila, surové výsledky každého zdroje s úplným původem a sloučený logický dokument. Všechny vrstvy se staly volitelnými. Parametry každého kroku vytěžování se uchovávají, aby ladicí nástroj mohl vzít `vim.json`, najít zdrojový obrázek a krok zopakovat se stejným nastavením. A každá změna hodnoty po vytěžení, ať od operátora, pravidla nebo jazykového modelu, se zapisuje jako **revize**, zatímco původní surová hodnota se nikdy nemění.

Rozhodnutí končilo větou, na které tomuto repozitáři záleží: schémata nového formátu patří do `api/contract/schemas/`, jako jazykově nezávislé kontrakty v JSON Schema.

Dva jazyky, jeden model

V květnu měl parser zhruba 20 000 řádků Javy ve třinácti modulech Mavenu a pythonový svět reco potřeboval stejnou logiku nativně. Volat Javu z každého pythonového nasazení znamenalo JVM vedle každého workeru a startovací režii u každého dokumentu. Přepsat 20 000 řádků ručně nebylo pro jednoho architekta reálné.

ADR-007, přijaté 8. května 2026, zvolilo třetí cestu: překladový nástroj. Nový modul projektu Dream, `dream-java2py`, čte zdrojový kód Javy skutečným parserem jazyka Java, mapuje knihovny (PDFBox, ZXing, Lucene) na jejich pythonové protějšky a nechává jazykový model psát idiomatický Python, soubor po souboru, přičemž každý soubor projde rukama člověka, než přistane v podbalíčku `ptx_reco.vim`. Port roste postupně a nástroj vede záznam o tom, co je přeloženo a co ještě čeká.

Stejný měsíc přinesl ještě dvě věci. Dne 15. května vznikla samostatná **výzkumná laboratoř** parseru, kde se problémy vytěžování sbírají jako dlouhodobé položky katalogu a měří ve verzovaných výzvách. A 19. května vznikl javový model vrstvy `extraction`; další den vyšel parser ve verzi 0.1.8.

Žebřík a nová vrstva

Nové druhy informací stále přicházely a tým si všiml, že o každém se rozhodovalo zvlášť: čárové kódy se staly kolekcí ve fyzické vrstvě, obsah knihy samostatnou vrstvou. Dne 9. října 2026 proměnilo ADR-009 tyto zvyklosti v pravidlo: **rozhodovací žebřík**. Nová informace se nejdřív zkusí stát volitelným polem existujícího prvku, pak kolekcí v existující vrstvě a teprve jako poslední krok novou vrstvou, přes tři brány a čtyři kritéria. Jedna z dalších kapitol žebřík podrobně prochází.

Téhož dne jím prošel první kandidát. ADR-010 přidalo vrstvu `prescriptive` a verzi **2.1**. Drží to, co dokument *předepisuje*: požadavky, doporučení a svolení specifikace, odkazy na jiné dokumenty a pojmy, které definuje. Jazykový model ustanovení navrhne, člověk je potvrdí nebo zamítne a každé ustanovení ukazuje na stránku a blok, ze kterého pochází.

Téhož dne dorazila schémata nové vrstvy do tohoto repozitáře. Během dalších dvou dnů dostal model preskriptivní vrstvy javový parser i pythonový port a javový model extrakční vrstvy se srovnal s kontraktem, vadu po vadě. Dne 16. října zpřísnil kontrakt vazbu mezi vrstvami a verzemi: dokument s vrstvou `prescriptive` musí říkat `"2.1"`, dokument s vrstvou `extraction` musí říkat `"2.0"` nebo `"2.1"`. `vim_version` řady 2.x je slib, už ne seznam funkcí.

Verze v kostce

Verze	Co přidala	Poprvé
1.0	Textové bloky: fyzická a strukturální vrstva	30. ledna 2026
1.1	Čárové kódy	30. ledna 2026
1.2	Metadata dokumentu a plný text	31. ledna 2026
1.3	Stemming	31. ledna 2026
1.4	Vrstva obsahu	8. března 2026

Verze	Co přidala	Poprvé
1.5	Grafika, tabulky a grafické prvky; výchozí verze parseru	8. března 2026
2.0	Vrstva `extraction`, všechny vrstvy volitelné, zdroje, revize	specifikována 24. března 2026
2.1	Vrstva `prescriptive`, pole `property` u revizí	specifikována 9. října 2026

Prvním commitem a verzí 2.1 dělí devět měsíců. Za tu dobu se VIM změnil z pluginu, který čte faktury, v model sdílený javovým parserem, pythonovým portem, frameworkem pro rozpoznávání formulářů, pipeline pro vytěžování faktur a kontraktem v tomto repozitáři. Další kapitola otevírá skutečný `vim.json` a dívá se, co je uvnitř.

Prameny

- Historie commitů `plugins/vim-parser` v `portunix-plugins` (30. ledna až 11. října 2026)
- ADR-004 *VIM as Universal Result Container* (24. března 2026), ADR-007 *Port vim-parser to Python via Dream Translation Tool* (8. května 2026), ADR-009 *Criteria for Adding a New VIM Layer* a ADR-010 *VIM Prescriptive Layer* (9. října 2026), `portunix-architecture`
- CHANGELOG.md tohoto repozitáře: preskriptivní vrstva VIM (#42) a vazba vrstev na `vim_version` (#51)
- Historie commitů `src/ptx_reco/vim` v `portunix-reco` a repozitáře `portunix-vim-parser-research-lab`

ČÁST 3

Uvnitř VIM

Co obsahuje vim.json, stroj, který ho píše, a kontrakt, který drží všechny výrobce i čtenáře v poctivosti.

4 · Uvnitř vim.json

5 · Stroj, který čte

6 · Slib v JSON Schema

KAPITOLA 4

Uvnitř vim.json

Malá faktura

Abychom mohli nahlédnout dovnitř VIM, potřebujeme dokument. Pro tuto knihu jsme jeden vyrobili: jednostránkovou fakturu vymyšlené sklářské firmy *Sklo a vrstvy s.r.o.* vymyšlené odběratelce v Brně. Má nadpis, dva adresní bloky vedle sebe, datum vystavení, malou tabulku se dvěma položkami, celkovou částku a QR kód pro platbu ve formátu SPAYD. Každé jméno a každé číslo je smyšlené, takže faktura smí být otištěna ve veřejné knize.

PDF jsme předali skutečnému parseru, verzi 0.1.8, s výchozím nastavením:

```
java -jar vim-parser-0.1.8-SNAPSHOT.jar auto -o faktura-2026-0042.pdf.vim.json faktura-2026
-0042.pdf
```

Výsledkem je soubor JSON o zhruba 31 kilobajtech. Všechno v této kapitole je z něj okopírované, jen dlouhá pole jsou zkrácena. PDF, `vim.json` i skript, který fakturu vyrobil, leží vedle knihy, takže si běh můžete zopakovat.

Obálka

Každý dokument VIM má nahoře stejné tři části:

```
{
  "vim_version" : "1.4",
  "document" : {
    "source" : "faktura-2026-0042.pdf",
    "pages" : 1,
    "metadata" : {
      "encodingFormat" : "application/pdf",
      "contentSize" : 11122,
      "inLanguage" : "cs",
      "producer" : "WeasyPrint 70.0"
    }
  },
  "layers" : { "physical" : { }, "structural" : { }, "semantic" : { }, "toc" : { } }
}
```

`vim_version` říká `"1.4"`, protože dokument má vrstvu obsahu; předchozí kapitola tento zvyk řady 1.x vysvětlila. `document` popisuje zdroj: název souboru, počet stran a metadata se jmény převzatými ze slovníku Schema.org (`encodingFormat`, `inLanguage`). Jazyk parser rozpoznal jako češtinu sám. A `layers` nese skleněné tabule.

Fyzická vrstva: co tam je

Fyzická vrstva naší faktury má pět textových bloků, jeden čárový kód, jednu tabulku, jeden grafický prvek a plný text. Blok je skupina textových prvků, které stojí blízko sebe. Takto začíná blok s odběratelem:

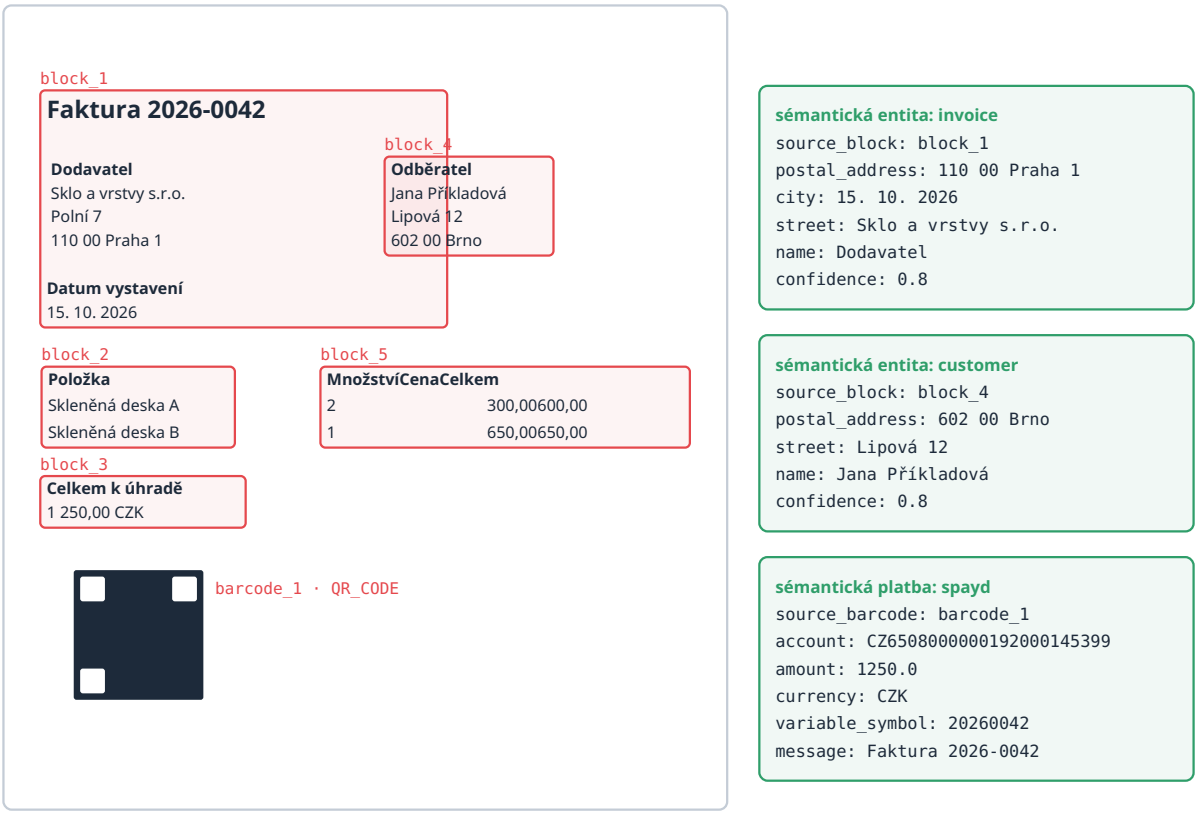
```
{
  "id" : "block_4",
  "bbox" : { "x" : 299.1, "y" : 155.7, "width" : 84.4, "height" : 44.8 },
  "page" : 1,
  "text_elements" : [ {
    "text" : "Odběratel",
    "font_size" : 14.0,
    "font_name" : "UWGIWA+DejaVu-Sans-Bold",
    "confidence" : 1.0
  }, {
    "text" : "Jana Příkladová",
    "font_name" : "XKQQSR+DejaVu-Sans"
  } ]
}
```


Každý prvek zná svůj obdélník na stránce (`bbox`, v bodech PDF), svou stránku a své písmo. Název písma končí u popisku na *Bold* a u jména ne; právě tento drobný detail parseru později umožní popisky najít. Spolehlivost je 1,0, protože text převzatý přímo z PDF se nehádá. Textový prvek přečtený OCR ze skenu by nesl nižší číslo.

Čárový kód je uložen přesně tak, jak byl dekódován, i se svou polohou:


```
{
  "id" : "barcode_1",
  "page" : 1,
  "format" : "QR_CODE",
  "raw_content" : "SPD*1.0*ACC:CZ6508000000192000145399*AM:1250.00*CC:CZK*X-
VS:20260042*MSG:Faktura 2026-0042"
}
```


Tady pracuje princip obousměrnosti. Fyzická vrstva obsahuje dost na to, aby se stránka dala znovu nakreslit. Následující obrázek nakreslil skript jen z `vim.json`: každé slovo stojí tam, kde ho parser viděl, každý červený rámeček je fyzický blok a zelené rámečky vpravo ukazují, co z nich udělala sémantická vrstva.



Obrázek 4.1 Vzorová faktura překreslená z fyzické vrstvy, s bloky a výsledky sémantické vrstvy

Strukturální vrstva: jak to vypadá

Strukturální vrstva dává každému fyzickému bloku tvar. ID zůstávají stejná:

```
{
  "id" : "block_4",
  "type" : "labeled_content",
  "significance" : "primary",
  "page" : 1,
  "header" : "Odběratel",
  "content" : [ "Jana Příkladová", "Lipová 12", "602 00 Brno" ]
}
```

Tučný první řádek se stal záhlavím (`header`), zbytek obsahem (`content`). Přesně tuto strukturu jazykový model v prologu v prostém textu postrádal. Strukturální záznam dostane i čárový kód: typ obsahu (`spayd`) a seznam bloků ve svém okolí, aby čtenář našel text, který kód vysvětluje.

Sémantická vrstva: co to znamená

Na sémantické vrstvě mění parser bloky v entity a kódy v platby:

```
"entities" : [ {
  "type" : "customer",
  "source_block" : "block_4",
  "fields" : { "name" : "Jana Příkladová", "street" : "Lipová 12", "postal_address" : "602
00 Brno" },
  "confidence" : 0.8
} ],
"payments" : [ {
  "type" : "payment",
  "source_barcode" : "barcode_1",
  "format" : "spayd",
  "fields" : { "account" : "CZ6508000000192000145399", "amount" : 1250.0,
               "currency" : "CZK", "variable_symbol" : "20260042", "message" : "Faktura
2026-0042" },
  "confidence" : 1.0
} ]
```


Stojí za to všimnout si dvou věcí. Zaprvé každý sémantický výsledek ukazuje na svůj zdroj: `source_block` nebo `source_barcode`. Zadruhé se spolehlivosti liší. Zákazník byl nalezen porovnáním popisku *Odběratel* se známými záhlavími, což je heuristika, a tak dostal 0,8. Platba byla dekódována ze standardního formátu se všemi povinnými poli, a tak dostala 1,0.

Kde se parser spletl

Poctivá kniha musí ukázat i druhou polovinu. Tentýž soubor obsahuje tři chyby a každá něco učí.

Nadpis spolkl dodavatele. Blok `block_1` obsahuje nadpis *Faktura 2026-0042*, adresu dodavatele i datum vystavení, protože stojí v jednom sloupci blízko sebe. Strukturální vrstva proto vzala nadpis jako záhlaví celého bloku a sémantická vrstva vytvořila entitu typu `invoice`, jejíž `name` je *Dodavatel* a jejíž `city` je *15. 10. 2026*. Spolehlivost je přesto 0,8: heuristika neví, že se nechala napálit.

Sloupce tabulky se slily. Ve fyzickém bloku tabulky se buňky *300,00* a *600,00* staly jedním řetězcem `300,00600,00`, protože parser spojil znaky na jednom řádku bez mezery mezi sloupci. Detektor tabulek, který pracuje samostatně, našel mřížku o dvanácti řádcích a čtyřech sloupcích přes velkou část stránky, s buňkami jako *Faktura 202* a *6-004*. Plný text z extraktoru metadat má naopak řádky tabulky správně: *Skleněná deska A 2 300,00 600,00*.

Obsah našel nadpisy ve faktuře. Vrstva `toc` uvádí jako položky nadpis a tři záhlaví bloků se zdrojem `heuristic`. U faktury je to šum; u knihy by to bylo užitečné.

Nic z toho není skryté. Každá chybná hodnota nese ID bloku, ze kterého pochází, metodu a spolehlivost a spotřebitel ji může prověřit. Vícesloupcová sazba a tabulky jsou přesně ty problémy, které výzkumná laboratoř parseru vede jako položky katalogu, jak ukáže poslední kapitola.

Extrakční vrstva: co nese formulář

Naše faktura extrakční vrstvu nemá; parser ji nevyrábí. Kontrakt v tomto repozitáři má ale její malý příklad, `form-extraction.vim.json`: jeden naskenovaný formulář, jedno pole. Myšlenku revizí ukazuje lépe než jakýkoli popis:

```
{
  "field_id" : "order_number",
  "label" : "Order number",
  "value" : "A-1007",
  "raw_value" : "A-I007",
  "data_type" : "string",
  "confidence" : 0.71,
  "revisions" : [ {
    "revision" : 1,
    "old_value" : "A-I007",
    "new_value" : "A-1007",
    "actor" : "rule",
    "actor_id" : "rule:digit_lookalikes",
    "reason" : "character_replacement"
  } ]
}
```

OCR engine přečetl velké *I* tam, kde má formulář číslici *1*. Pravidlo, které zná podobně vypadající znaky, to opravilo. `raw_value` uchovává, co engine skutečně přečetl, a nikdy se nemění; `value` ukazuje aktuální stav; revize říká, kdo hodnotu změnil a proč. Pokud ji později opraví operátor, přibude druhá revize.



Obrázek 4.2 Jedno pole extrakční vrstvy: surová hodnota enginu, revize pravidlem a aktuální hodnota

Kolem polí drží vrstva odkaz na šablonu, která definovala zóny (`template_ref`), jeden záznam pro každý zdroj s jeho původem (`sources`) a sloučený logický dokument (`document`), kde se scházejí pole z mnoha stránek i tabulky rozdělené přes stránky.

Preskriptivní vrstva: co předepisuje

Druhý příklad kontraktu, `definition-file.vim.json`, ukazuje nejmladší vrstvu. Jeho zdrojem je dvanáctistránkový dokument s pravidly pro rohová razítka technických výkresů. Jedno z jeho ustanovení:

```
{
  "provision_id" : "p-5.3.2-1",
  "kind" : "requirement",
  "clause" : "5.3.2",
  "text" : "The title block shall contain the drawing number.",
  "anchors" : [ { "page" : 7, "element_type" : "block", "element_id" : "block_41" } ],
  "relations" : [ { "provision_id" : "p-5.3.1-1", "relation" : "refines" } ],
  "verification" : { "method" : "automatic", "subject" : "drawing",
                     "rule" : { "field" : "drawing_number", "check" : "present" } },
  "status" : "confirmed",
  "confidence" : 0.82
}
```

Ustanovení je požadavek, doporučení nebo svolení. Má stabilní ID, číslo článku, doslovný text a kotvu do fyzické vrstvy téhož dokumentu: strana 7, blok 41. Může zpřesňovat jiné ustanovení, záviset na něm, být z něj výjimkou nebo s ním být v rozporu. Říká, jak se dá ověřit: automaticky, testem nebo člověkem. A má stav. Jazykový model navrhuje ustanovení jako `proposed`; teprve člověk je udělá `confirmed`. V příkladu historie revizí ukazuje, že recenzent změnil druh tohoto ustanovení z *doporučení* na *požadavek* s komentářem *The verbal form is shall.*

Šest vrstev v jedné tabulce

Vrstva	Hlavní obsah	Typický výrobce
`physical`	`blocks` s textovými prvky, `barcodes`, `tables`, `graphics`, `fulltext`	vim-parser
`structural`	`blocks` se záhlavím, obsahem, typem a důležitostí; `barcodes` s bloky v okolí	vim-parser
`semantic`	`entities`, `payments`, `stemming`	vim-parser
`toc`	`entries` s nadpisem, stránkou, úrovní, blokem a zdrojem	vim-parser
`extraction`	`template_ref`, `sources`, `document`; pole s `revisions`	reco a další
`prescriptive`	`definition`, `provisions`, `references`, `terms`	výrobce s jazykovým modelem, kontrolovaný člověkem

Další kapitola se dívá na stroj, který píše první čtyři vrstvy: jak PDF projde pluginy parseru a vyjde z něj `vim.json`.

KAPITOLA 5

Stroj, který čte

Dílna, ne jeden stroj

Dobrým obrazem `vim-parseru` není jeden velký stroj s jedním vstupem a jedním výstupem. Je to dílna se specialisty. Jeden čte metadata souboru. Další rozřezává text na bloky. Další hledá čárové kódy, další tabulky, další obrázky. Když specialista na čárové kódy skončí, přečte specialista na platby kódy, které našel. Nakonec někdo složí z nadpisů obsah. Každý specialista umí jednu práci a své výsledky nechává na společném stole pro ostatní.

V kódu jsou specialisté **pluginy** a společný stůl je **kontext pluginů**. Vedoucím dílny je **orchestrátor**, který rozhoduje, kdo kdy pracuje.

Pluginy v balíčcích

Parser je projekt v Javě 21 s více než tuctem modulů Mavenu. Jádrem je jediný „tlustý“ JAR s příkazovou řádkou, orchestrátorem a běhovým prostředím pluginů. Každý plugin je samostatný soubor ZIP v adresáři `plugins/` vedle něj. Běhovým prostředím pluginů je PF4J, malý framework pro pluginy v Javě: při startu najde soubory ZIP, každý načte vlastním class loaderem a zeptá se ho, co umí.

Tento návrh přišel druhý den života parseru a nebyl náhodou. Plugin lze přidat, vyměnit nebo odebrat bez nového sestavení jádra. Nativní backendy lemmatizátoru pro češtinu, psané v C++, žijí ve vlastních pluginech; nasazení, které je nepotřebuje, prostě nemá jejich soubory ZIP.

Pluginy jsou několika druhů, daných třemi body rozšíření v modulu API:

Druh	Bod rozšíření	Práce	Pluginy
Extraktor	`ExtractorPlugin`	Čte surová data ze souboru	metadata, fulltext, barcode, graphics, table, lemmatizer
Parser	`ParserPlugin`	Interpretuje data, která našly extraktory	spayd
Vizualizér	`VisualizerPlugin`	Vytváří pohled na hotový `vim.json`	html

Plugin obsahu je také extraktor, ale běží ve vlastní fázi, *transform*, protože staví na textu, který našly jiné pluginy.

Kdo pracuje kdy

Každý plugin deklaruje, které pluginy potřebuje, než může začít. Plugin čárových kódů nepotřebuje nic. Plugin SPAYD potřebuje čárové kódy. Obsah potřebuje plný text. Orchestrátor tyto deklarace přečte, sestaví graf závislostí a promění ho v plán spuštění, fázi po fázi. O plán se dá požádat, aniž by se cokoli spustilo. Pro vzorovou fakturu z předchozí kapitoly parser odpoví:

```
VIM Parser - Execution Plan
===========================
Input: faktura-2026-0042.pdf (10,9 KB, 1 page)

Discovered plugins: 8
Auto-enabled: 7

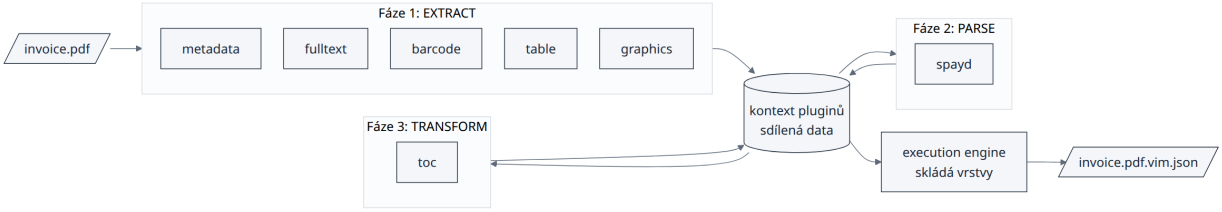
Execution sequence:
  Phase 1: EXTRACT
    [1] metadata     (no dependencies)
    [2] fulltext     (no dependencies)
    [3] barcode      (no dependencies)
    [4] table        (no dependencies)
    [5] graphics     (requires: barcode)

  Phase 2: PARSE
    [6] spayd        (requires: barcode)

  Phase 3: TRANSFORM
    [7] toc          (requires: fulltext)

Excluded from auto mode:
    [-] lemmatizer   (autoEnabled=false)
```

To je výstup `auto --dry-run` verze 0.1.8, okopírovaný beze změn. Lemmatizátor je nainstalovaný, ale ve výchozím stavu vypnutý; běží jen tehdy, když o stemming požádáte volbou `--stemming`. To je význam slova *auto* v příkazu: bez jakékoli konfigurace spustí orchestrátor každý plugin, který je ve výchozím stavu zapnutý, v pořadí, které respektuje všechny závislosti.



Obrázek 5.1 Cesta PDF parserem: extraktory, parsery a fáze transformace nechávají výsledky ve sdíleném kontextu a engine z nich skládá vrstvy

Společný stůl

Pluginy se nikdy nevolají navzájem. Mluví spolu přes sdílená data kontextu pluginů, mapu ze jmen na hodnoty. Extraktor textových bloků uloží své bloky pod `textBlocks`, extraktor čárových kódů své kódy pod `barcodes`, parser SPAYD přečte `barcodes` a vrátí zpět `spaydPayments`. Ve výchozím stavu běží pluginy jeden po druhém; konfigurační soubor `vim-parser.json` může přepnout režim spouštění na `thread` a pak extraktory, které na sobě nezávisejí, běží souběžně. V obou případech je sdílená mapa jediné místo, kde se potkávají.

Když jsou všechny fáze hotové, vezme **execution engine** hodnoty z mapy a sestaví `VimDocument`: fyzické bloky jdou do fyzické vrstvy, z každého fyzického bloku se odvodí strukturální blok, platby jdou do sémantické vrstvy, obsah do vlastní vrstvy. Dva kroky se odehrávají jen tady, až po pluginech: sémantické entity se hledají porovnáním záhlaví bloků se známými popisky a každý čárový kód dostane seznam textových bloků ve svém okolí. Tady se také nastavuje `vim_version` podle funkcí, které dokument používá, onen zvyk popsaný ve druhé kapitole.

Vidět, co viděl stroj

Parser, který o jednostránkové faktuře napíše 31 kilobajtů JSON, potřebuje způsob, jak se na výsledek mohou podívat lidé. Vizualizér `html` promění `vim.json` v jedinou HTML stránku se záložkami pro strukturální, fyzickou, sémantickou vrstvu a vrstvu obsahu, s barevně odlišenými bloky a přepínačem, který skryje pomocné bloky, jako jsou čísla stránek. Pracuje nad hotovým souborem, takže funguje pro jakéhokoli výrobce VIM, nejen pro tento parser:

```
portunix vim-parser visualize faktura-2026-0042.pdf.vim.json
```

Pro lidi, kteří parser udržují, existuje i druhý druh zraku: od března 2026 umí parser posílat trasy OpenTelemetry, takže čas každého pluginu nad každým dokumentem lze sledovat v nástroji pro trasování.

Týž model v Pythonu

Druhá kapitola vyprávěla, proč byl parser přenesen do Pythonu. Tady je, jak port vypadá zevnitř. Podbalíček `ptx_reco.vim` v `portunix-reco` zrcadlí moduly Javy: balíček `api` s rozhraními pluginů a kontextem pluginů, balíček `core` s execution enginem, orchestrací a registrem pluginů, `extractors` pro čárové kódy, plný text, grafiku, lemmatizátor, tabulky a obsah a `parsers` se SPAYD. Modelové třídy, v Javě vnořené, se staly pythonovými dataclass na úrovni modulu.

Co čím nahradit, rozhoduje mapa knihoven překladu: PDFBox, ZXing, Tika a Lucene ustupují svým pythonovým protějškům a Jackson dataclass s výslovnou serializací do JSON. Port má i něco, co javový parser nemá: extraktor OCR, který z obrazu naskenované stránky udělá fyzické bloky s původem, protože reco čte skeny.

Port je důvodem, proč má VIM dnes dvě implementace a proč každá nová vrstva musí přijít s plánem pro obě, jak vysvětlí další kapitola. Pythonová strana už píše vrstvy `extraction` a `prescriptive`; zpět je zatím nečte a pipeline reco extrakční vrstvu zatím neplní. Kontrakt jmenuje reco výrobcem této vrstvy; dokumentace portu poctivě říká, že jde o plánovaný stav, ne o současný.

Spotřebitel v provozu: faktury do ISDOC

Nejjasnějším příkladem VIM jako sdíleného modelu je `portunix-ireco`, pipeline pro polostrukturované dokumenty, jako jsou faktury. Její první krok, `parse`, spouští execution engine z `ptx_reco.vim` s extraktorem textových bloků a podle možností s extraktory čárových kódů, tabulek, grafiky, obsahu a metadat. Výsledkem je `vim.json` s fyzickou a strukturální vrstvou. Druhý krok, `extract`, ten soubor přečte, najde pole faktury pomocí šablon s pružným rozvržením a zapíše je do sémantické vrstvy téhož souboru. Třetí krok umí výsledek vyexportovat do ISDOC, českého standardu XML pro elektronické faktury.

```
ptx-ireco parse invoice.pdf -o invoice.vim.json     # PDF → VIM (physical + structural)
ptx-ireco extract invoice.vim.json                  # fills the semantic layer in place
```

Dva nástroje, dva kroky, jeden soubor. Každý krok přidá své vrstvy a ostatní nechá být. K tomu byly skleněné tabule z první kapitoly.

Prameny

- `plugins/vim-parser` v `portunix-plugins`: README modulů, `Main.java`, výstup `auto --dry-run` a `plugins --deps` verze 0.1.8 (25. října 2026)
- `src/ptx_reco/vim/README.md` v `portunix-reco`
- `README.md` repozitáře `portunix-ireco`
- ADR-007 *Port vim-parser to Python via Dream Translation Tool*, `portunix-architecture`

KAPITOLA 6

Slib v JSON Schema

Dvě implementace, jedna pravda

Jakmile měl VIM dvě implementace, javový parser a pythonový port, objevilo se tiché nebezpečí. Dva týmy mohou číst tutéž specifikaci a pochopit ji trochu jinak. Pole zapsané na jedné straně jako `rawValue` a na druhé jako `raw_value`; číslo, které jedna strana píše vždy a druhá jen tehdy, když není prázdné; řetězec verze, který jedna strana kontroluje a druhá přehlíží. Každý rozdíl je malý. Dohromady dají soubor, který jeden program napíše a druhý nepřečte.

Odpovědí ekosystému je **kontrakt**: popis formátu, který není kódem v žádném jazyce, který umí zkontrolovat stroj a který žije na jednom místě. Pro VIM je tím místem tento repozitář, adresář `contract/schemas/`. Rozhodlo o tom ADR-004 v březnu 2026. V říjnu 2026 s ním obě implementace srovnaly své modely a javový parser dostal test, který porovnává jeho model preskriptivní vrstvy se schématy.

Co kontrakt obsahuje

Kontrakt VIM je sada souborů JSON Schema v dialektu Draft 2020-12:

Schéma	Co popisuje
`vim-document.schema.json`	Kořen: `vim_version`, `document`, `layers`
`vim-extraction.schema.json`	Extrakční vrstvu: odkaz na šablonu, zdroje, sloučený dokument
`vim-extraction-source.schema.json`	Jeden zdroj vytěžování s jeho původem
`vim-extraction-field.schema.json`	Jedno vytěžené pole s hodnotou, surovou hodnotou a revizemi
`vim-prescriptive.schema.json`	Preskriptivní vrstvu: definici, odkazy, pojmy
`vim-prescriptive-provision.schema.json`	Jedno ustanovení s kotvami, vztahy a ověřením

Většinu váhy nesou tři pravidla kořenového schématu.

Objekt `layers` je uzavřený. Zná přesně šest klíčů, od `physical` po `prescriptive`, a každý jiný odmítne. Výrobce si nemůže vymyslet vlastní vrstvu; nová vrstva je změnou kontraktu, o které se rozhoduje veřejně.

`vim_version` je seznam známých hodnot. Přijímá verze od `"1.0"` do `"2.1"` a nic jiného.

Vrstvy jsou svázané s verzemi. Dokument s vrstvou `prescriptive` musí říkat `"2.1"`. Dokument s vrstvou `extraction` musí říkat `"2.0"` nebo `"2.1"`. Toto pravidlo přišlo 16. října 2026 a bylo označeno jako nekompatibilní změna kontraktu: dokumenty, které měly novou vrstvu se starým číslem verze, dříve prošly a nyní jsou odmítnuty. Skončil tím zvyk řady 1.x, kdy verze byla seznamem funkcí. V řadě 2.x je verze slibem o tom, jaké vrstvy může čtenář najít.

Příklady, které musí projít, a příklady, které musí selhat

Schéma, které se nikdy netestuje, je jen přání. Kontrakt proto přichází s příklady a pravidla tohoto repozitáře říkají, že každé schéma potřebuje alespoň jeden. Pro VIM existují dva platné příklady, oba už v této knize zazněly: `form-extraction.vim.json` s jedním naskenovaným formulářem a `definition-file.vim.json` s definičním souborem a jeho ustanoveními.

Vedle nich leží v adresáři `contract/examples/invalid/` čtyřicet dokumentů VIM, které musí **selhat**. Každý porušuje právě jedno pravidlo: kotva na straně nula, spolehlivost větší než jedna, ustanovení bez ID, odkaz s neznámým rozsahem, klíč vrstvy, který neexistuje, extrakční vrstva s verzí `"1.5"`. Když někdo pravidlo omylem uvolní, jeden z těchto dokumentů najednou projde a kontrola selže.

Kontrola je jeden příkaz:

```
make validate-contract
```

Najde každý příklad, vybere schéma podle přípony názvu souboru (`.vim.json` jde na `vim-document.schema.json`) a ohlásí každý výsledek. Několik řádků skutečného běhu:

```
PASS  [vim-document.schema.json] contract/examples/definition-file.vim.json
PASS  [vim-document.schema.json] contract/examples/form-extraction.vim.json
PASS  [vim-document.schema.json] contract/examples/invalid/anchor-page-zero.vim.json
(invalid as expected: /layers/prescriptive/provisions/0/anchors/0/page must be >= 1)
PASS  [vim-document.schema.json] contract/examples/invalid/prescriptive-wrong-vim-
version.vim.json (invalid as expected: /vim_version must be equal to constant)
...
98 checked, 0 failed, 8 skipped (no matching contract suffix).
```

Co JSON Schema neřekne

Některá pravidla jsou mimo dosah JSON Schema. Umí říct, že ustanovení má ID; neumí říct, že žádná dvě ustanovení nemají stejné ID. Umí říct, že ustanovení vyjmenovává odkazy; neumí říct, že každý vyjmenovaný odkaz ve stejné vrstvě existuje.

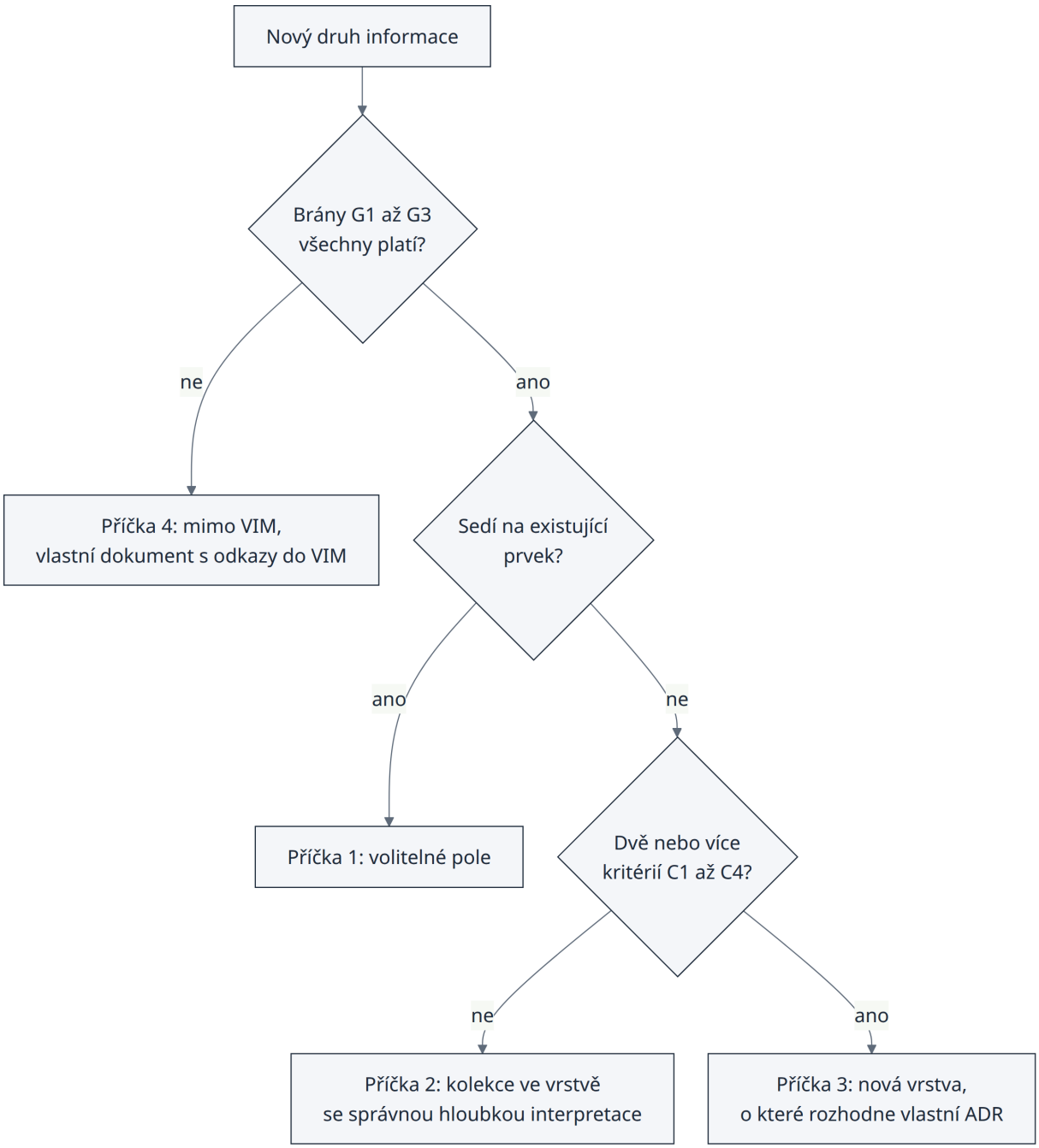
Pro tato pravidla má repozitář malou kontrolu navíc, `scripts/vim-identifier-check.mjs`, která běží po kontrole schématem nad každým příkladem VIM. Ověří, že `provision_id`, `reference_id` a `term_id` jsou ve vrstvě jedinečné a že každý odkaz, pojem a vztah ustanovení ukazuje na něco, co existuje. Její funkce `checkVimIdentifiers()` je exportovaná, takže ji jiné projekty mohou importovat a uplatnit stejná pravidla na vlastní dokumenty. Ve výše uvedeném běhu tu selže příklad `provision-reference-without-target.vim.json` se zprávou *no reference with reference_id 'r-9'*.

Kdy je něco novou vrstvou?

Uzavřený objekt `layers` otevírá otázku, kterou dříve či později potká každý formát: když přijde nová informace, kam patří? Čárové kódy se staly kolekcí ve fyzické vrstvě. Obsah knihy se stal vlastní vrstvou. O každém případu se rozhodovalo zvlášť a ADR-009 si všimlo, že špatné rozhodnutí je drahé oběma směry. Zbytečná vrstva potřebuje schéma tady, oddíl ve specifikaci, model v

Javě, model v Pythonu, podporu ve vizualizéru a zlaté testovací soubory: jen javový model extrakční vrstvy má 25 tříd a každá vrstva se platí dvakrát. Chybějící vrstva zase vtlačí cizí model do vrstvy s jiným významem.

Odpovědí je **rozhodovací žebřík**. Nová informace zvolí první příčku, která sedí:



Obrázek 6.1 Rozhodovací žebřík ADR-009: nová vrstva VIM je poslední příčka, až po třech branách a alespoň dvou ze čtyř kritérií

Tři **brány** rozhodují, zda informace do VIM vůbec patří:

ID	Brána
G1	Je výsledkem porozumění souboru: odvozená z dokumentu, bez něj nedává smysl

ID	Brána
G2	Každá položka může nést původ: zdroj, stránku, oblast nebo ID jiného prvku
G3	Je užitečná i jinému spotřebiteli než svému výrobci

Čtyři **kritéria** rozhodují, zda potřebuje vlastní vrstvu; alespoň dvě musí být „ano“:

ID	Kritérium
C1	Má jiného výrobce nebo jiný životní cyklus než výstup vim-parseru
C2	Vyroste ve vlastní doménový model s několika typy entit a vztahy
C3	Bude mít revize nebo stav: hodnoty se po vytvoření mění
C4	Kotví do více než jedné vrstvy nebo druhu prvků

Dvě malá pravidla drží počítání poctivé. Když jsou revize jediným důvodem jiného životního cyklu, počítají se C1 a C3 jako jedno „ano“. A na kritéria se odpovídá pro příštích dvanáct měsíců, ne pro každou možnou budoucnost: „možná později“ je „ne“.

Žebřík byl vyzkoušen na historii a výsledek je poučný. Vrstva `extraction` získá tři „ano“ a vrstvou by se stala znovu. Platby SPAYD nezískají žádné a jsou právem kolekcí. A vrstva `toc` také nezíská žádné: podle těchto pravidel by byla kolekcí ve strukturální vrstvě. Vrstvou zůstává, protože přesun by bez užitku rozbil všechny existující čtenáře. ADR to říká otevřeně a dodává, že `toc` není precedentem.

Prvním kandidátem, který vyšplhal celý žebřík, byla vrstva `prescriptive`, která odpověděla „ano“ na všechny tři brány i všechna čtyři kritéria. Její ADR také ukazuje, co musí každá nová vrstva přinést: je volitelná, na jiné vrstvy odkazuje přes ID, místo aby je kopírovala, její klíč je jedno anglické podstatné jméno malými písmeny, její schéma přichází **před** jakýmkoli kódem a má plán pro obě implementace.

Kontrakt je slib cizím lidem

Na kontraktech tohoto repozitáře závisejí jiné projekty a mnoho jejich autorů se s autory VIM nikdy nepotká. Proto jsou zdejší pravidla ke změnám přísná: každé schéma má vlastní verzi (`$version`), každá nekompatibilní změna se zapisuje do changelogu tučně a každá změna schématu přichází s příklady, které projdou, i s příklady, které selžou. Kontrakt je slib cizím lidem. JSON Schema je jen jazyk, ve kterém je napsán.

Další část knihy opouští nitro VIM a ukazuje, jak ho používat: jak vyrobit `vim.json`, prohlédnout si ho, zkontrolovat a číst z vlastního kódu.

Prameny

- contract/README.md a contract/schemas/README.md tohoto repozitáře
- scripts/vim-identifier-check.mjs a výstup `make validate-contract` (25. října 2026)
- CHANGELOG.md: #42 a #51
- ADR-004 a ADR-009, `portunix-architecture`

ČÁST 4

Práce s VIM

Od PDF ke zkontrolovanému vim.json několika příkazy a otevřené otázky, které tvarují další verze.

7 · Váš první vim.json

8 · Kam VIM směřuje

KAPITOLA 7

Váš první vim.json

Od čtení k práci

Pět kapitol vyprávělo příběh VIM a nahlíželo do jeho nitra. Tato je pro vaše ruce. Na jejím konci budete mít z PDF vyrobený `vim.json`, prohlédnete si ho v prohlížeči, položíte mu dvě otázky krátkým programem v Pythonu a zkontrolujete ho proti kontraktu. Každý příkaz a každý výstup v této kapitole proběhl nad vzorovou fakturou knihy, která leží vedle kapitol v adresáři `samples/`.

Potřebujete Javu 21 pro parser, Python 3.11 nebo novější pro příklady a volitelně `jq`, malý nástroj příkazové řádky pro JSON.

Krok 1: Zeptejte se na plán

Parser se distribuuje jako adresář s jedním souborem JAR a adresářem `plugins/` se soubory ZIP. Uvnitř Portunixu běží tentýž parser jako `portunix vim-parser`; mimo něj voláte JAR přímo. Než cokoli spustíte, zeptejte se parseru, co by udělal:

```
java -jar vim-parser-0.1.8-SNAPSHOT.jar auto --dry-run faktura-2026-0042.pdf
```

Odpovědí je plán spuštění, který jste viděli v kapitole o stroji, který čte: sedm pluginů ve třech fázích a lemmatizátor vynechaný, protože ve výchozím stavu není zapnutý. Plán je levný způsob, jak zjistit, které pluginy parser našel. Chybí-li v seznamu nějaký plugin, chybí jeho soubor ZIP v `plugins/`.

Krok 2: Vyrobte soubor

```
java -jar vim-parser-0.1.8-SNAPSHOT.jar auto -o faktura-2026-0042.pdf.vim.json faktura-2026
-0042.pdf
```

S volbou `-o` jde výsledek do zadaného souboru; bez ní na standardní výstup. Volba `-w` ho zapíše vedle PDF jako *sidecar* soubor s příponou `.vim.json`. Nejužitečnější další volby příkazu `auto` jsou:

Volba	Výchozí	Účinek
`--metadata` / `--no-metadata`	zapnuto	Metadata dokumentu se jmény ze Schema.org
`--fulltext`	vypnuto	Plný text dokumentu ve fyzické vrstvě
`--hash`	vypnuto	Otisk SHA-256 souboru v metadatech
`--stemming`	vypnuto	Kmeny slov v sémantické vrstvě, pro vyhledávání
`--backend <id>`, `--lang <code>`	lucene, auto	Backend a jazyk stemmingu
`--exclude <plugins>`	nic	Pluginy, které se vynechají, oddělené čárkou
`--compact`	vypnuto	JSON bez odsazení

Krok 3: Prohlédněte si ho

`vim.json` jedné stránky má stovky řádků. Než je začnete číst, podívejte se na obrázek:

```
java -jar vim-parser-0.1.8-SNAPSHOT.jar visualize faktura-2026-0042.pdf.vim.json
```

Vizualizér vypíše jedinou HTML stránku se záložkou pro každou vrstvu; s volbou `-o` nebo `-w` ji zapíše do souboru. Otevřete ji v prohlížeči, přepněte na strukturální záložku a najděte blok se záhlavím *Odběratel*. Pak přepněte na fyzickou záložku a najděte tentýž blok podle jeho ID. Právě děláte ručně to, co každý spotřebitel VIM dělá v kódu: sledujete ID z vyšší vrstvy dolů k důkazu.

Krok 4: Ptejte se ho přes jq

Pro rychlý pohled z příkazové řádky stačí `jq`. Záhlaví všech strukturálních bloků:

```
jq -r '.layers.structural.blocks[] | "\(.id)\t\(.header)"' faktura-2026-0042.pdf.vim.json
```

```
block_1 Faktura 2026-0042
block_2 Položka
block_3 null
block_4 Odběratel
block_5 MnožstvíCenaCelkem
```

Výstup ukazuje na první pohled silné i slabé stránky ze třetí kapitoly: blok odběratele má své záhlaví, celková částka žádné a záhlaví tabulky se slilo do jednoho slova.

Krok 5: Ptejte se ho z kódu

`vim.json` je obyčejný JSON, takže ke čtení není potřeba žádná zvláštní knihovna. Tento krátký program odpovídá na otázku z prologu, *kdo je zákazník*, a přidává důkaz: stránku a polohu bloku, ze kterého odpověď pochází. Pak přečte platbu:

```
import json
from pathlib import Path

doc = json.loads(Path("faktura-2026-0042.pdf.vim.json").read_text(encoding="utf-8"))
layers = doc["layers"]

# Who is the customer, and where on the page does it come from?
for entity in layers.get("semantic", {}).get("entities", []):
    if entity["type"] == "customer":
        block = next(b for b in layers["physical"]["blocks"] if b["id"] == entity["source_b
lock"])
        print(entity["fields"]["name"], "| page", block["page"], "| bbox", block["bbox"]["x
"], block["bbox"]["y"],
              "| confidence", entity["confidence"])

# What is there to pay?
for payment in layers.get("semantic", {}).get("payments", []):
    f = payment["fields"]
    print(f["amount"], f["currency"], "to", f["account"], "VS", f["variable_symbol"])
```

```
Jana Příkladová | page 1 | bbox 299.13782 155.6819 | confidence 0.8
1250.0 CZK to CZ6508000000192000145399 VS 20260042
```

Všimněte si vzoru, protože je to vzor každého dobrého spotřebitele VIM. Význam čte ze sémantické vrstvy, po `source_block` sestupuje do fyzické vrstvy pro důkaz a spolehlivost drží vedle odpovědi. Program, který spolehlivost cestou zahodí, udělá z kvalifikovaného odhadu falešný fakt.

Všechny vrstvy jsou volitelné, a tak program používá u každé vrstvy `.get()` s prázdnou výchozí hodnotou. `vim.json` z rozpoznávání formulářů nemusí mít sémantickou vrstvu vůbec.

Krok 6: Zkontrolujte ho proti kontraktu

Kontrakt z páté kapitoly umí kontrolovat i vaše vlastní soubory, nejen příklady tohoto repozitáře. Poslouží jakýkoli validátor JSON Schema Draft 2020-12, pokud načte všechny soubory `vim-*.schema.json`, protože kořenové schéma odkazuje na ostatní přes jejich `$id`. V Pythonu s knihovnami `jsonschema` a `referencing`:

```
import json
import sys
from pathlib import Path

from jsonschema import Draft202012Validator
from referencing import Registry, Resource

schemas = Path(sys.argv[1])
registry = Registry().with_resources(
    (s["$id"], Resource.from_contents(s))
    for s in (json.loads(p.read_text(encoding="utf-8")) for p in schemas.glob("vim-
*.schema.json"))
)
root = json.loads((schemas / "vim-document.schema.json").read_text(encoding="utf-8"))
validator = Draft202012Validator(root, registry=registry)
doc = json.loads(Path(sys.argv[2]).read_text(encoding="utf-8"))
errors = sorted(validator.iter_errors(doc), key=lambda e: list(e.path))
for e in errors:
    print("/" + "/".join(map(str, e.path)), e.message)
print("valid" if not errors else f"{len(errors)} error(s)")
```

Pro vzorovou fakturu vypíše `valid`. Pro jeden z negativních příkladů kontraktu, extrakční vrstvu s příliš starou verzí, vypíše:

```
/vim_version '1.5' is not one of ['2.0', '2.1']
1 error(s)
```

Má-li váš dokument preskriptivní vrstvu, spusťte po kontrole schématem ještě kontrolu identifikátorů; `checkVimIdentifiers()` ve `scripts/vim-identifier-check.mjs` najde duplicitní ID a odkazy na ustanovení, odkazy a pojmy, které neexistují.

Krok 7: Dva nástroje, jeden soubor

Poslední krok ukazuje VIM jako sdílený kontejner. Pipeline faktur `portunix-ireco` pracuje dvěma příkazy nad jedním souborem:

```
ptx-ireco parse invoice.pdf -o invoice.vim.json     # PDF → VIM (physical + structural)
ptx-ireco extract invoice.vim.json                  # fills the semantic layer in place
```

První příkaz zapíše fyzickou a strukturální vrstvu pythonovým portem parseru. Druhý je přečte a přidá sémantickou vrstvu, aniž by sáhl na to, co už tam je. Váš vlastní nástroj se může do takového řetězu přidat stejně: přečíst vrstvy, které potřebuje, zapsat vrstvu, kterou vlastní, a zbytek nechat být.

Vyzkoušejte si

1. Spusťte parser nad vzorovou fakturou s volbou `--exclude barcode`. Co se stane a co musíte k volbě přidat, abyste dostali `vim.json` bez čárových kódů?
2. Upravte program v Pythonu tak, aby vypsal dodavatele. Co vypíše a co vám to říká o sémantické vrstvě tohoto vzorku?
3. Napište ručně `vim.json` jen s vrstvou `extraction` a s `vim_version "1.5"`. Co řekne validátor? Co musíte změnit?
4. Najděte v této kapitole větu, která vysvětluje, proč má spotřebitel spolehlivost uchovat. Pak najděte ve vzorku hodnotu, jejíž spolehlivost je na to, co říká, příliš vysoká.

Odpovědi

1. Parser skončí dřív, než cokoli přečte: *Plugin 'spayd' depends on missing plugin 'barcode'*. Orchestrátor pluginy závislé na vynechaném pluginu potichu nevyřadí; nechá rozhodnutí na vás. Na čárových kódech závisí i extraktor grafiky, takže fungující volba je `--exclude barcode,spayd,graphics`. Fyzická vrstva pak má prázdné `barcodes` a `graphics` a sémantická vrstva prázdné `payments`. Takovou změnu si nejdřív ověřte přes `--dry-run`.
2. Entita typu `supplier` neexistuje. Adresa dodavatele skončila v `block_1` spolu s nadpisem a entita vytvořená z tohoto bloku má typ `invoice` a jméno *Dodavatel*. Sémantická vrstva je heuristika a na tomto rozvržení selhává.
3. `/vim_version '1.5' is not one of ['2.0', '2.1']`. Extrakční vrstva existuje od verze 2.0, takže dokument musí říkat `"2.0"` nebo `"2.1"`.
4. Entita typu `invoice` z `block_1` má spolehlivost 0,8, stejnou jako správný zákazník, přestože její `city` je datum. Spolehlivost heuristiky říká, jak moc si je heuristika jistá, ne jak moc má pravdu.

Poslední kapitola se dívá dopředu: co VIM ještě neumí dobře a kam směřuje.

KAPITOLA 8

Kam VIM směřuje

Poctivý střed

Každý mladý formát žije v poctivém středu. Je dost dobrý na to, aby se používal každý den, a ještě ne dost dobrý na to, aby se na něj zapomnělo. VIM je teď přesně tam. Vzorová faktura této knihy ukázala obě poloviny na jedné stránce: zákazníka a platbu přečtené správně, i s důkazy, a nadpis, který spolkl dodavatele, a tabulku, jejíž sloupce se slily. Formát unesl oba druhy výsledků stejně dobře. Samotné čtení má ale ještě dlouhou cestu před sebou.

Tato poslední kapitola se na tu cestu dívá: na problémy, které jsou známé, na místo, kde se zkoumají, na otázky, které architektonická rozhodnutí záměrně nechala otevřené, a na myšlenku, která to všechno drží pohromadě.

Laboratoř pro těžké stránky

V květnu 2026 založil tým pro výzkum parseru samostatný repozitář, `portunix-vim-parser-research-lab`. Má neobvyklé pravidlo: parser nikdy nemění přímo. Drží ho jako submodul jen pro čtení, pouští ho nad sbírkami dokumentů, porovnává ho s jinými nástroji a svá zjištění předává týmu parseru jako issue.

Laboratoř pracuje se dvěma druhy záznamů. **Položka katalogu** popisuje známý problém vytěžování a žije dlouho; neuzavírá se jen proto, že skončil jeden pokus. **Výzva** je jedno měření: verzované, opakovatelné srovnání s jasnou podmínkou vítězství a písemným verdiktem. První tři položky katalogu jsou přesně ty problémy, na které dříve či později narazí každý, kdo čte PDF:

ID	Problém	Co se pokazí
I-PDF-001	Zápatí stránek	Čísla stránek, živá záhlaví a řádky s copyrightem se objevují v textu na každém zlomu stránky
I-PDF-002	Vícesloupcová sazba	Text čtený v uloženém pořadí kličkuje mezi sloupci
I-PDF-003	Tabulky přes zlom stránky	Z jedné logické tabulky jsou dvě a řádek záhlaví je tam dvakrát

První výzva, C001, porovnává `vim-parser` s nástrojem Marker, open source převodníkem z PDF do Markdownu, právě na těchto třech problémech. V době psaní je to kostra: pipeline je definovaná, běhy ještě nezačaly. To není slabina laboratoře. Je to její metoda: nejdřív se dohodnout, co se bude měřit a co se počítá jako vítězství, a teprve potom měřit.

Slité sloupce naší faktury patří do rodiny I-PDF-002 a tabulka, kterou detektor našel přes velkou část stránky, do rodiny I-PDF-003. Vzorek z této knihy by se mohl stát malou položkou té sbírky.

Otázky nechané otevřené záměrně

Dobrá architektonická rozhodnutí nepředstírají, že vědí všechno. ADR o VIM končí otevřenými otázkami a vyplatí se je vyjmenovat, protože ukazují, kudy bude formát růst.

Místo pro pokusy. Objekt `layers` je uzavřený a nová vrstva potřebuje schéma a dvě implementace. To chrání spotřebitele, ale nenechává místo pro obsah, který ještě není dost stabilní na kontrakt. ADR-009 zmiňuje možnou *oblast rozšíření* pro takový obsah a nechává ji jako samostatné rozhodnutí.

Registr vrstev. Při šesti vrstvách se seznam vrstev s jejich vlastníky a výrobci pohodlně vejde do specifikace formátu. Při více vrstvách může být potřeba krátký registr.

Jazyk pro pravidla. Ustanovení preskriptivní vrstvy může říct, že se ověřuje automaticky, a nese objekt `rule`. Co smí v tomto objektu stát, kontrakt neurčuje; nástroje se na tom musí dohodnout mimo něj. Malý jazyk pravidel je otevřenou otázkou ADR-010, spolu s formátem dokumentu hodnocení, který zaznamená výsledek kontroly souladu.

Domov výrobce s jazykovým modelem. Preskriptivní vrstvu píše výrobce, který používá jazykový model. Kde tento výrobce žije a jak se spouští, je také stále otevřené; ADR-010 navrhuje, aby nástroje, které používají jazykový model během vývoje, běžely jako skilly Claude Code.

Druhá polovina pythonového portu. Pythonový model vrstvy `extraction` a `prescriptive` píše, ale zpět je nečte, a rozpoznávací pipeline reco extrakční vrstvu zatím neplní. Dokud to nedělá, jmenuje kontrakt výrobce, který je spíš plánovaný než skutečný, a dokumentace to přiznává.

Drobné pravdy pro velké modely

V době velkých jazykových modelů svádí myšlenka, že na struktuře už nezáleží. Model přečte PDF, nebo dokonce fotografii stránky, a odpoví na otázky o ní. Proč držet tři skleněné tabule, ID a spolehlivosti, když se model může prostě podívat?

Pokus z prologu je první odpovědí. Tentýž model nad tímtéž dokumentem si s prostým textem nebyl jistý a s vrstvami jistý byl. Struktura není berlička pro slabé čtenáře; je to způsob, jak silný čtenář ukazuje svou práci. Druhou odpovědí je původ. Model, který odpoví *Jana Příkladová*, je užitečný. Systém, který odpoví *Jana Příkladová, strana 1, blok 4, spolehlivost 0,8, nalezeno podle popisku Odběratel*, se dá ověřit, opravit a dá se mu věřit. Když se ponoříme do čtení dokumentů, odhalíme pestrý svět drobných, trpělivých dovedností: seskupování, popisování, měření, pochybování. Snadno je přehlédneme, když se ženeme za novými cíli. VIM je uchovává, a to ve tvaru, který přečte člověk, program i jazykový model.

Preskriptivní vrstva ukazuje na třetí odpověď. Tam je jazykový model pozván jako výrobce a formát mu poctivě dělá místo: každé ustanovení, které navrhne, začíná jako `proposed` a teprve člověk ho udělá `confirmed`. Model přináší rychlost, vrstvy přinášejí důkazy, člověk přináší poslední slovo.

Zpátky ke sklu

Začínali jsme třemi skleněnými tabulemi nad stránkou. O devět měsíců později jich je šest a další mohou přibýt, každá přes brány a kritéria žebříku. Myšlenka pod nimi se nezměnila: uchovat, co tam je, uchovat, jak je to seskupené, uchovat, co to znamená, a nechat každou hodnotu říct, odkud pochází a jak moc si jsme jistí.

Dokument, který prošel VIM, umí říct, co znamená. To byl slib jména a na tom slibu budou pracovat i další verze.

Prameny

- `README.md`, `docs/catalog/pdf/` a `challenges/C001-pdf-vs-marker/` repozitáře `portunix-vim-parser-research-lab`
- ADR-009 a ADR-010, oddíl *Open Questions*, `portunix-architecture`
- `src/ptx_reco/vim/README.md` v `portunix-reco`